SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:
Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

1.Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

Coding Challenge: Star Topology Using Python

Aim:

To develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program accepts the number of hosts and the Cat6 UTP cable length for each host, displays the topology in ASCII format, verifies that all hosts are connected through the central switch, and explains the reliability of star topology in a LAN.

Python Code:

```
# Star Topology Generator

print("=====================================")
print("       STAR TOPOLOGY GENERATOR")
print("=====================================")

# Get number of hosts
n = int(input("Enter the number of hosts: "))

# Central switch
switch = "Central Switch"

hosts = []

# Get cable length for each host
for i in range(1, n + 1):
    length = float(input(f"Enter Cat6 UTP cable length for Host-{i} (in meters): "))
    hosts.append((f"Host-{i}", length))

print("\n=====================================")
```

```
print("          STAR TOPOLOGY")
print("=====================================")

print("\n                  [Central Switch]")
print("                     / | \\")

# Display connections
for host, length in hosts:
    print(f"    {host} -------- {length} m -------- [Central Switch]")

print("\n=====================================")
print("        CONNECTION DETAILS")
print("=====================================")

# Display host, switch and cable length
for host, length in hosts:
    print(f"{host} -> {switch} -> Cat6 UTP Cable: {length} m")

# Verify all hosts are connected
if len(hosts) == n:
    print("\nVerification: SUCCESS")
    print(f"All {n} hosts are connected through the central switch.")
else:
    print("\nVerification: FAILED")
    print("Some hosts are not connected.")

print("\n=====================================")
print("        COMMUNICATION EXPLANATION")
print("=====================================")

print("""
In a star topology, every host is directly connected to a
central switch. The switch manages communication between
the connected hosts.

If one host or its cable fails, the other hosts can continue
communicating because they have separate connections to
```

the central switch.

Therefore, star topology provides reliable communication, easy fault detection, and simple network management in a Local Area Network (LAN).
""")

Sample Input:

```
====================================
      STAR TOPOLOGY GENERATOR
====================================
Enter the number of hosts: 5

Enter Cat6 UTP cable length for Host-1 (in meters): 10
Enter Cat6 UTP cable length for Host-2 (in meters): 15
Enter Cat6 UTP cable length for Host-3 (in meters): 20
Enter Cat6 UTP cable length for Host-4 (in meters): 12
Enter Cat6 UTP cable length for Host-5 (in meters): 18
```

Sample Output:

```
====================================
        STAR TOPOLOGY
====================================

                [Central Switch]
                   / | \

  Host-1 -------- 10.0 m -------- [Central Switch]
  Host-2 -------- 15.0 m -------- [Central Switch]
  Host-3 -------- 20.0 m -------- [Central Switch]
  Host-4 -------- 12.0 m -------- [Central Switch]
  Host-5 -------- 18.0 m -------- [Central Switch]

====================================
      CONNECTION DETAILS
====================================
```

Host-1 -> Central Switch -> Cat6 UTP Cable: 10.0 m
Host-2 -> Central Switch -> Cat6 UTP Cable: 15.0 m
Host-3 -> Central Switch -> Cat6 UTP Cable: 20.0 m
Host-4 -> Central Switch -> Cat6 UTP Cable: 12.0 m
Host-5 -> Central Switch -> Cat6 UTP Cable: 18.0 m

Verification: SUCCESS
All 5 hosts are connected through the central switch.

Explanation:

In a Star Topology, all hosts are connected individually to a central switch. The switch acts as the central communication point and forwards data between the connected hosts.

The topology provides reliable communication because the failure of one host or its cable does not affect the other hosts. It also makes fault detection and network maintenance easier, since each host has a separate connection to the switch.

However, the central switch is a single point of failure. If the switch fails, communication between all connected hosts will be interrupted.

Result:

Thus, the Python program successfully generates a Star Topology, displays each host with its corresponding Cat6 UTP cable length, verifies that all hosts are connected through the central switch, and demonstrates how star topology supports reliable communication in a LAN.

2.Write a Python function named validate_segment(length_m) to verify whether a given UTP cable segment satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of 100 meters. The function should return True for valid cable lengths and return False along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least three different test cases and interpret the results.

Aim:

To write a Python function validate_segment(length_m) that checks whether a UTP cable segment complies with the IEEE 802.3 maximum cable length of 100 meters, and to test the function with different input values.

Python Code:

```
def validate_segment(length_m):
    if length_m <= 100:
        return True, "Valid cable length"
    else:
        return False, "Error: Cable length exceeds the maximum limit of 100 meters"



# Test cases
test_cases = [50, 100, 120]

for length in test_cases:
    result, message = validate_segment(length)
    print("Cable length:", length, "meters")
    print("Result:", result)
    print("Message:", message)
    print()
```

Sample Input

50

100

120

Sample Output:

Cable length: 50 meters

Result: True

Message: Valid cable length

Cable length: 100 meters

Result: True

Message: Valid cable length

Cable length: 120 meters

Result: False

Message: Error: Cable length exceeds the maximum limit of 100 meters

Explanation:

- The function validate_segment(length_m) accepts the cable length in meters.
- If the length is less than or equal to 100 meters, it returns True, indicating that the cable is valid.
- If the length is greater than 100 meters, it returns False with an appropriate error message.
- Three test cases are used:
    1. 50 m → Valid
    2. 100 m → Valid, because 100 m is the maximum permitted length
    3. 120 m → Invalid because it exceeds 100 m

Result:

Thus, the Python program successfully validates UTP cable segment lengths according to the 100-meter maximum limit. The test results confirm that lengths up to 100 meters are accepted, while lengths exceeding 100 meters are rejected with an appropriate error message.

3.Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named csma_log.txt, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

Aim:

To develop a Python program to simulate the CSMA/CD protocol in a star topology with four hosts connected through a central switch. The program simulates transmission attempts, detects collisions, calculates random back-off intervals, and records all events in csma_log.txt.

Python Code:

```
import random
import time

# Four hosts connected to a central switch
hosts = ["Host1", "Host2", "Host3", "Host4"]

# Maximum number of transmission attempts
MAX_ATTEMPTS = 5

# Log file
log_file = "csma_log.txt"


def log_message(message):
    """Write message to console and log file."""
    print(message)
    with open(log_file, "a") as file:
```

```
        file.write(message + "\n")


def transmit(host):
    """Simulate CSMA/CD transmission for a host."""

    attempt = 1

    while attempt <= MAX_ATTEMPTS:

        log_message(
            f"{host}: Transmission attempt {attempt}"
        )

        # Randomly determine whether a collision occurs
        collision = random.choice([True, False])

        if collision:
            log_message(
                f"{host}: Collision detected!"
            )

            # Binary Exponential Backoff
            backoff_slots = random.randint(0, (2 ** attempt) - 1)

            log_message(
                f"{host}: Back-off interval = "
                f"{backoff_slots} slot(s)"
            )
```

```
            # Simulate waiting
            time.sleep(0.5)

            attempt += 1

        else:
            log_message(
                f"{host}: Frame transmitted successfully."
            )
            break

    if attempt > MAX_ATTEMPTS:
        log_message(
            f"{host}: Transmission failed after "
            f"{MAX_ATTEMPTS} attempts."
        )


# Clear the previous log
open(log_file, "w").close()

log_message("===== CSMA/CD Simulation Started =====")
log_message("Topology: 4 Hosts connected through a Central Switch")
log_message("")

# Simulate transmissions from all hosts
for host in hosts:
    transmit(host)

log_message("")
```

log_message("===== CSMA/CD Simulation Completed =====")

Sample Input:

Host1

Host2

Host3

Host4

Sample Output:

Since the program uses random values, the output may differ each time.

===== CSMA/CD Simulation Started =====

Topology: 4 Hosts connected through a Central Switch

Host1: Transmission attempt 1

Host1: Collision detected!

Host1: Back-off interval = 1 slot(s)

Host1: Transmission attempt 2

Host1: Frame transmitted successfully.

Host2: Transmission attempt 1

Host2: Frame transmitted successfully.

Host3: Transmission attempt 1

Host3: Collision detected!

Host3: Back-off interval = 2 slot(s)

Host3: Transmission attempt 2

Host3: Frame transmitted successfully.

Host4: Transmission attempt 1

Host4: Frame transmitted successfully.

===== CSMA/CD Simulation Completed =====

Sample csma_log.txt:

===== CSMA/CD Simulation Started =====

Topology: 4 Hosts connected through a Central Switch

Host1: Transmission attempt 1

Host1: Collision detected!

Host1: Back-off interval = 1 slot(s)

Host1: Transmission attempt 2

Host1: Frame transmitted successfully.

Host2: Transmission attempt 1

Host2: Frame transmitted successfully.

Host3: Transmission attempt 1

Host3: Collision detected!

Host3: Back-off interval = 2 slot(s)

Host3: Transmission attempt 2

Host3: Frame transmitted successfully.

Host4: Transmission attempt 1

Host4: Frame transmitted successfully.

===== CSMA/CD Simulation Completed =====

Explanation:

1. **Star Topology:**

   Four hosts (Host 1 to Host 4) are connected to a central switch

2. **Central Sense:**

   A host attempts to transmit a frame when it is ready to send data.

3. **Collision Detection:**

   The program randomly simulates whether a collision occurs during transmission.

4. **Collision Handling:**

   When a collision occurs, the host stops the transmission and calculates a random back-off interval.

5. **Binary Exponential Backoff:**

   The waiting range increases after repeated collisions:

   - Attempt 1 → 0 to 1 slots
   - Attempt 2 → 0 to 3 slots
   - Attempt 3 → 0 to 7 slots

6. **Retransmission:**

   After the back-off period, the host attempts to transmit the frame again.

7. **Logging:**

   Every transmission attempt, collision, and back-off interval is stored in csma_log.txt.

Result:

The Python program successfully simulates the CSMA/CD protocol, including transmission attempts, collision detection, random back-off intervals, and retransmission. The csma_log.txt file records all important events. The simulation demonstrates how random backoff and collision detection help reduce repeated transmission conflicts in Ethernet networks.

4.Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

Aim:

To develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The program simulates MAC address learning, frame reception, frame forwarding, and displays the updated MAC address table after each successful frame transmission.

Python Code:

1. Server / Switch Program:

Save as server.py.

```
import socket

# Simulated MAC Address Table
mac_table = {}

HOST = "127.0.0.1"
PORT = 5000

# Create server socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((HOST, PORT))
server_socket.listen(1)

print("Central Switch is running...")
print("Waiting for client connection...")

conn, address = server_socket.accept()
```

```
print("Client connected:", address)

while True:
    data = conn.recv(1024).decode()

    if not data:
        break

    # Frame format: SOURCE_MAC,DESTINATION_MAC,MESSAGE
    source_mac, destination_mac, message = data.split(",", 2)

    # Learn source MAC address
    mac_table[source_mac] = "Port 1"

    print("\nFrame received by Switch")
    print("Source MAC      :", source_mac)
    print("Destination MAC :", destination_mac)
    print("Data            :", message)

    # Check destination MAC
    if destination_mac in mac_table:
        destination_port = mac_table[destination_mac]

        print("Destination found in MAC table.")
        print("Forwarding frame to:", destination_port)

        response = "Frame forwarded successfully"

    else:
        print("Destination MAC not found.")
        print("Switch floods the frame to other ports.")

        # Simulate learning the destination host
        mac_table[destination_mac] = "Port 2"

        response = "Frame forwarded successfully"
```

```
    # Display MAC Address Table
    print("\nUpdated MAC Address Table")
    print("-------------------------")
    print("MAC Address\t\tPort")

    for mac, port in mac_table.items():
        print(mac, "\t", port)

    # Send confirmation to client
    conn.send(response.encode())

conn.close()
server_socket.close()
```

2. Client / Host Program:

Save as client.py

```
import socket

HOST = "127.0.0.1"
PORT = 5000

# MAC addresses of two simulated hosts
SOURCE_MAC = "AA:BB:CC:DD:EE:01"
DESTINATION_MAC = "AA:BB:CC:DD:EE:02"

# Create client socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

client_socket.connect((HOST, PORT))

print("Connected to the central switch.")

# Send multiple frames
messages = [
    "Hello Host 2",
    "Data Frame 1",
```

```
    "Data Frame 2"
]

for message in messages:

    frame = SOURCE_MAC + "," + DESTINATION_MAC + "," + message

    print("\nSending frame:")
    print(frame)

    client_socket.send(frame.encode())

    response = client_socket.recv(1024).decode()

    print("Switch response:", response)

client_socket.close()

print("\nConnection closed.")
```

Sample Input:
The client sends the following frames:
Source MAC: AA:BB:CC:DD:EE:01
Destination MAC: AA:BB:CC:DD:EE:02

Message 1: Hello Host 2
Message 2: Data Frame 1
Message 3: Data Frame 2

Sample Output – Client:
Connected to the central switch.

Sending frame:
AA:BB:CC:DD:EE:01,AA:BB:CC:DD:EE:02,Hello Host 2
Switch response: Frame forwarded successfully

Sending frame:

AA:BB:CC:DD:EE:01,AA:BB:CC:DD:EE:02,Data Frame 1
Switch response: Frame forwarded successfully

Sending frame:
AA:BB:CC:DD:EE:01,AA:BB:CC:DD:EE:02,Data Frame 2
Switch response: Frame forwarded successfully

Connection closed.

Sample Output – Server/Switch
Central Switch is running...
Waiting for client connection...
Client connected: ('127.0.0.1', 54321)

Frame received by Switch
Source MAC : AA:BB:CC:DD:EE:01
Destination MAC : AA:BB:CC:DD:EE:02
Data : Hello Host 2

Destination found in MAC table.
Forwarding frame to: Port 2

Updated MAC Address Table

MAC Address Port
AA:BB:CC:DD:EE:01 Port 1
AA:BB:CC:DD:EE:02 Port 2

Frame received by Switch
Source MAC : AA:BB:CC:DD:EE:01
Destination MAC : AA:BB:CC:DD:EE:02
Data : Data Frame 1

Destination found in MAC table.
Forwarding frame to: Port 2

Updated MAC Address Table

MAC Address Port

AA:BB:CC:DD:EE:01 Port 1

AA:BB:CC:DD:EE:02 Port 2

Explanation:

- MAC Learning: The switch learns the source MAC address of each incoming frame and stores it with the corresponding port in the MAC address table.
- Frame Forwarding: The switch checks the destination MAC address. If it is found in the table, the frame is forwarded only to the associated port.
- Unknown MAC: If the destination MAC is not found, the switch floods the frame to other ports.
- MAC Table: The table is updated whenever the switch learns a new MAC address.
- Star Topology: All hosts communicate through the central switch, which manages frame forwarding.

Result:

The Python client-server application successfully simulates Ethernet switch operation in a star topology. The switch learns the source MAC address, stores it with the corresponding port, checks the destination MAC address, and forwards the frame through the appropriate port. The updated MAC Address Table is displayed after each successful frame transmission.

5.Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as add_port(host_id) to connect a host, remove_port(host_id) to disconnect a host, and broadcast(frame) to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

Aim:

To design and implement a Python class StarSwitch that simulates an Ethernet switch in a star topology by connecting and disconnecting hosts and broadcasting frames to all connected hosts except the source.

Python Code:

```
class StarSwitch:
    def __init__(self):
        self.active_ports = []
        self.log = []

    def add_port(self, host_id):
        self.active_ports.append(host_id)
        print(host_id, "connected to switch.")

    def remove_port(self, host_id):
        if host_id in self.active_ports:
            self.active_ports.remove(host_id)
            print(host_id, "disconnected from switch.")

    def broadcast(self, frame):
        source = frame["source"]
        message = frame["message"]

        destinations = []

        for host in self.active_ports:
```

```
            if host != source:
                destinations.append(host)

        print("\nBroadcast Frame")
        print("Source:", source)
        print("Message:", message)
        print("Received by:", destinations)

        self.log.append(
            f"Source: {source}, Destinations: {destinations}"
        )


# Create switch
switch = StarSwitch()

# Connect multiple hosts
switch.add_port("Host1")
switch.add_port("Host2")
switch.add_port("Host3")
switch.add_port("Host4")

# Broadcast frames
switch.broadcast({
    "source": "Host1",
    "message": "Hello everyone"
})

switch.broadcast({
    "source": "Host3",
    "message": "Network announcement"
})

# Disconnect a host
switch.remove_port("Host4")

# Broadcast again
```

```
switch.broadcast({
   "source": "Host2",
   "message": "Host4 is disconnected"
})
```

Sample Input:

Host1, Host2, Host3, Host4

Broadcast from Host1

Broadcast from Host3

Disconnect Host4

Broadcast from Host2

Sample Output:

Host1 connected to switch.

Host2 connected to switch.

Host3 connected to switch.

Host4 connected to switch.

Broadcast Frame

Source: Host1

Message: Hello everyone

Received by: ['Host2', 'Host3', 'Host4']

Broadcast Frame

Source: Host3

Message: Network announcement

Received by: ['Host1', 'Host2', 'Host4']

Host4 disconnected from switch.

Broadcast Frame

Source: Host2

Message: Host4 is disconnected

Received by: ['Host1', 'Host3']

Explanation:

- **add_port()** connects a host to the switch.

- **remove_port()** disconnects a host.
- **broadcast()** sends the frame to **all active hosts except the source**.
- The program maintains active_ports to keep track of connected hosts.
- Each broadcast is recorded in the log with the **source and destination hosts**.
- **Broadcasting** is important in Ethernet when the sender does not know the destination MAC address or when information needs to reach all devices in the network.

Result:

The StarSwitch class successfully simulates a basic Ethernet switch. It connects multiple hosts, broadcasts frames to active ports, excludes the source host, and records each broadcast operation.